

Subject: CS336 Lecture 2, PyTorch and Resource Accounting

Date: from 2026/7/26 to 2026/7/27

Contents

Knowledge Overview	1
Memory Accounting	1
张量与阶数	1
张量的内存	1
张量在 GPU 上	3
Compute Accounting	3
Einops	3
张量运算的 FLOPs	4
算术强度 (Arithmetic Intensity)	5
Memory and Compute Accounting for Training	6
深度网络	6
梯度基础	7
梯度的 FLOPs	7
优化器与训练循环	8
More Memory Optimizations	8
Summary: Resource Accounting	9

Knowledge Overview

本讲要带走的知识

- **Mechanics**（机制）. 直接明了，主要是 PyTorch 语义；
- **Mindset**（思维）. 资源核算（resource accounting）——记得去做；
- **Intuitions**（直觉）. 对资源如何被花掉形成感觉，今天没有 ML 魔法.

Memory Accounting

张量与阶数

张量：一切数据的基本单元

张量（Tensor）是用于存储一切数据的基本构建单元，包括数据、参数、梯度、优化器状态与激活值. 每个张量都有一个阶数（**rank**），即其维度的数量：`torch.zeros(4)` 是 rank-1（vector），`torch.zeros(4, 8)` 是 rank-2（matrix），`torch.zeros(4, 8, 2)` 是 rank-3.

在 Transformer 中常见的是 rank-4 张量：

```
B, S, H, D = 32, 16, 16, 64
x = torch.zeros(B, S, H, D)
```

其中 B 是 batch size, S 是 sequence length, H 是 head 数, D 是每个 head 的 hidden dimension.

张量的内存

内存由元素数量与数据类型决定

张量的元素通常是浮点数，内存由值的数量和每个值的数据类型共同决定. fp32 是默认类型，每个浮点数占 4 字节，因此一个 4×8 的张量占 128 字节：

```
x = torch.zeros(4, 8)
assert get_memory_usage(x) == 4 * 8 * 4
```

GPT-3 前馈层中的一个矩阵（`torch.empty(12288 * 4, 12288)`）约 2.3 GB.

fp32（单精度）

fp32（float32）是传统科学计算的基准，某些场景会用双精度 fp64. 在深度学习中，你可以“马虎”很多.

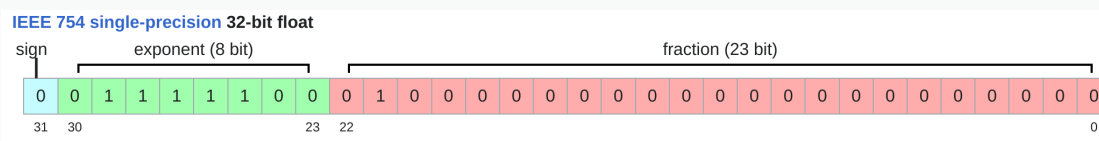


Figure 1: IEEE 754 单精度 32 位浮点的位布局.

fp16（半精度）

fp16 把每个元素降到 2 字节，但动态范围差，小数字会下溢（Underflow）：`torch.tensor([1e-8], dtype=torch.float16)` 会变成 0。如果训练时发生，可能导致不稳定。

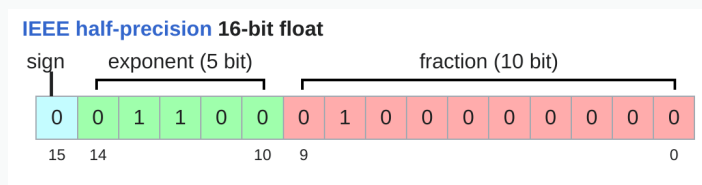


Figure 2: fp16 的位布局。

bf16（脑浮点）

Google Brain 在 2018 年提出 bfloat16：与 fp16 同样占 2 字节，但拥有与 fp32 相同的动态范围，因此 `torch.tensor([1e-8], dtype=torch.bfloat16)` 不会下溢。代价是精度（分辨率）较低，但在深度学习中影响不大。

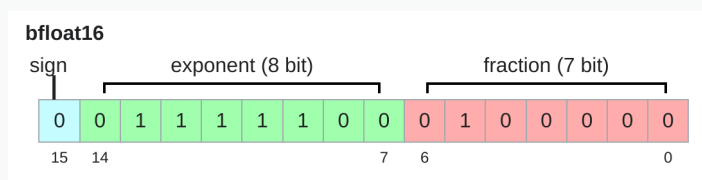


Figure 3: bf16 与 fp32 共享指数位，因此动态范围相同。

混合精度训练（Mixed precision）

纯 fp32 训练内存开销大，纯 fp16 / bf16 又有不稳定风险。解决方案（Micikevicius+ 2017）是混合精度：

- 参数、激活值和梯度用 bf16；
- 优化器状态用 fp32。

PyTorch 的自动混合精度（AMP）会在安全时（例如 `matmul`，而非 `exp`）自动转换：with `torch.amp.autocast("cuda", dtype=torch.bfloat16)`：....

fp8 与 fp4

- **fp8（2022）**。为 ML 工作负载标准化，H100 支持两种变体：**E4M3**（范围 $[-448, 448]$ ）与 **E5M2**（范围 $[-57344, 57344]$ ）；
- **nvfp4（2025）**。每个值仅占 4 位，可表示 -6, -4, -3, -2, -1.5, -1, -0.5, 0, 0.5, 1, 1.5, 2, 3, 4, 6；每个块用单独的缩放因子，因此实际动态范围更大（只是不能与邻近值自由变化）。Nemotron 3 Super 就是在 NVFP4 下训练的。

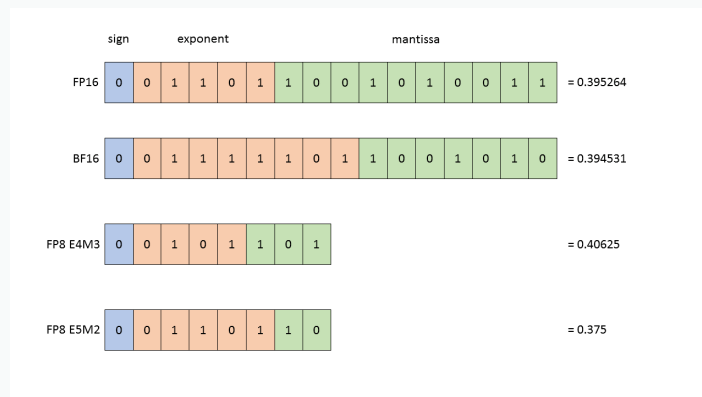


Figure 4: fp8 的两种格式 E4M3 与 E5M2.

张量在 GPU 上

CPU 与 GPU 内存

默认情况下张量存储在 CPU 内存中 (`x.device == torch.device("cpu")`). 为了利用 GPU 的大规模并行能力, 需要把张量搬到 GPU 内存: `x = x.to(device)`, 或用 `with torch.device(device)`: 直接在 GPU 上创建.

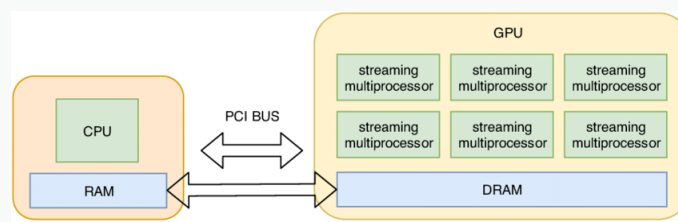


Figure 5: CPU 内存与 GPU 内存之间的 PCIe 传输.

Compute Accounting

Einops

Einops 与 einsum

Einops 是一个操作张量的库, 特点是**维度被命名**, 灵感来自爱因斯坦求和约定 (Einstein, 1916). Einsum 是广义矩阵乘法, 具有清晰的“记账”能力:

```
z = x @ y
z = einsum(x, y, "seq1 hidden, hidden seq2 -> seq1 seq2")
```

batch 维度需要广播时也一样:

```
z = einsum(x, y,
           "batch seq1 hidden, batch seq2 hidden -> batch seq1 seq2")
```

- 输出中没有指定的维度会被求和 (约减掉);

- 用... 表示对任意数量维度的广播.

reduce 与 rearrange

reduce. 对某个维度做归并（求和、平均、最大、最小），减少张量的总元素数量：

```
y = x.sum(dim=-1)
y = reduce(x, "... hidden -> ...", "sum")
```

rearrange. 有时一个维度实际代表两个维度（被压平在一起），需要对其中一个操作. 下面用 (3, 8) 的输入与 (4, 4) 的权重模拟“多头”线性变换：先拆成 2 个头，组内做线性变换，再合并回 8 维.

```
x = rearrange(x, "... (heads hidden1) -> ... heads hidden1", heads=2)
x = einsum(x, w, "... hidden1, hidden1 hidden2 -> ... hidden2")
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)")
```

形状依次为 (3, 8) → (3, 2, 4) → (3, 2, 4) → (3, 8).

张量运算的 FLOPs

FLOPs 与 FLOP/s

浮点运算（FLOP）是基本运算，如加法或乘法. 有两个发音相同、极易混淆的缩写：

- **FLOPs**：浮点运算次数，衡量完成的计算量；
- **FLOP/s**：每秒浮点运算次数，衡量硬件的速度.

直观数据. 训练 GPT-3 (2020) 约 3.14×10^{23} FLOPs, GPT-4 (2023) 据推测约 2×10^{25} FLOPs. H100 峰值在开启稀疏性时为 1979 万亿 FLOP/s, 不开启时约减半 ($h100_flop_per_sec = 1979 \times 10^{12}/2$).

线性模型的 FLOPs. 一个 $[B, D] \times [D, K]$ 的矩阵乘法，每个 (i, j, k) 三元组一次乘法加一次加法，因此

$$\text{flops} = 2BDK$$

实际 FLOP/s 就是 flops/time ；注意 FLOP/s 高度依赖数据类型. **MFU (Model FLOPs Utilization)** 定义为

$$\text{MFU} = \frac{\text{实际 FLOP/s}}{\text{峰值 FLOP/s}}$$

通常 $\text{MFU} \geq 0.5$ 就算相当不错. 为什么不能接近 1? 需要看计算在 GPU 上如何执行.

算术强度 (Arithmetic Intensity)

一次计算如何执行

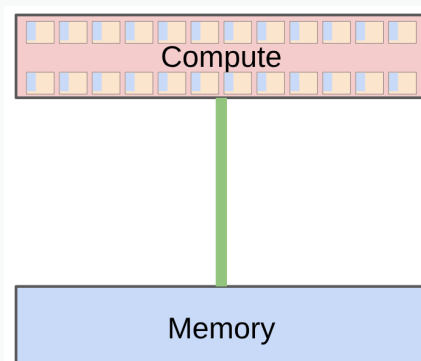


Figure 6: 计算单元与内存之间的数据搬运.

计算分三步：把输入从内存送到加速器、在加速器上执行、把输出送回内存. 耗时取决于加速器速度 (FLOP/s) 与内存带宽 (bytes/s). H100 上

```
h100_flop_per_sec = 1979e12 / 2  
h100_bytes_per_sec = 3.35e12
```

假设通信与计算完美重叠, 则总时间为 $\max(\text{通信时间}, \text{计算时间})$.

加速器强度与瓶颈判定

加速器强度是硬件的分界线: GPU 每从显存搬 1 byte, 至少要做多少 FLOPs 才能把计算单元也忙满.

$$\text{加速器强度} = \frac{\text{峰值 FLOP/s}}{\text{内存带宽}} \quad \text{算术强度} = \frac{\text{flops}}{\text{bytes}}$$

判定规则:

- 内存受限 (Memory-bound): 算术强度 < 加速器强度;
- 计算受限 (Compute-bound): 算术强度 > 加速器强度.

一般来说, 我们会发现自己常常处于内存受限的状态.

各类操作的算术强度

设规模为 n , 读入与写出都按 bf16 (2 字节) 计:

操作	bytes	flops	算术强度	瓶颈
ReLU	$4n$	n	$1/4$	内存
GELU	$4n$	$20n$	5	内存
点积	$4n + 2$	$2n - 1$	$\approx 1/2$	内存
矩阵-向量	$2n^2 + 4n$	$n(2n - 1)$	≈ 1	内存
矩阵-矩阵	$6n^2$	$n^2(2n - 1)$	$\approx n/3$	计算

几个要点：

- GELU 每字节做的计算比 ReLU 多，算术强度更高，但两者都仍是内存受限——所以单独执行时 ReLU 并不比 GELU 更快；
- 只要矩阵足够大就是计算受限：训练 Transformer 涉及大规模矩阵乘法，因此是计算受限；
- 矩阵-向量乘法出现在推理阶段，这就是推理是内存受限的原因；
- 算术 / 加速器强度也取决于精度（bf16 与 fp32 不同）。

屋顶线图与 MFU

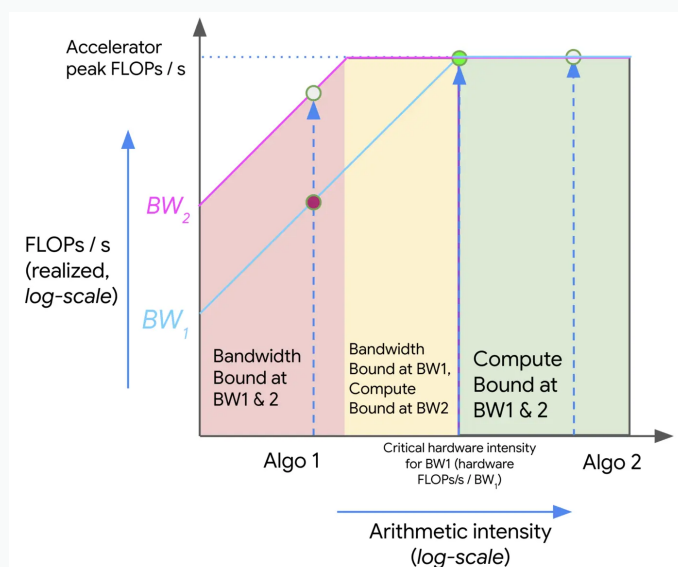


Figure 7: 屋顶线图：不同带宽下的内存受限区与计算受限区。

x 轴上每个切片代表一个计算操作（各有其算术强度），每段分段线性函数对应一种硬件，拐点（kink）就是加速器强度。于是可以把它与 MFU 联系起来：

$$MFU = \min\left(1, \frac{\text{算术强度}}{\text{加速器强度}}\right)$$

Memory and Compute Accounting for Training

深度网络

深度网络：L 层、D 维

考虑一个有 L 层、输入 / 激活 / 输出都是 D 维的网络，每层是线性变换加 ReLU。

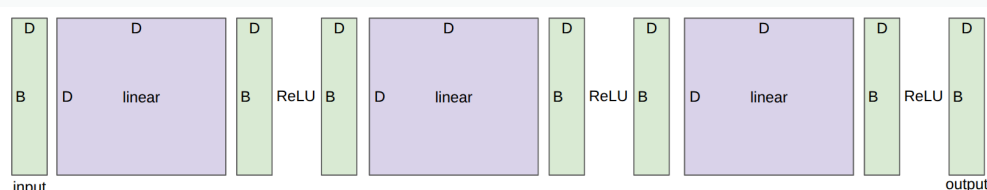


Figure 8: 深度网络：每层是线性变换加 ReLU.

每层有 $D \times D$ 个参数，因此参数量为 D^2L ；对整个 batch 前向传播的 FLOPs 为 $2BD^2L$.

梯度基础

前向传播与反向传播

以最简单的线性模型为例：

$$y = 0.5(x \cdot w - 5)^2$$

前向计算损失：pred_y = x @ w, loss = 0.5 * (pred_y - 5).pow(2). 反向调用 loss.backward(), 梯度存入 w.grad.

```
loss.backward()
assert torch.equal(w.grad, torch.tensor([1, 2, 3]))
```

梯度的 FLOPs

loss.backward() 做了什么

loss.backward() 是 PyTorch 自动求导的“总开关”，只做一件事：计算所有 requires_grad=True 的张量的梯度，并把结果存进各自的.grad 属性（本身不返回值）。内部是一条反向流水线：

1. 从 loss 开始，梯度初始为 1.0；
2. 反向到 h2，得到临时的 h2.grad（需 retain_grad() 才会保留）；
3. 用 h2.grad 与 w2 同时算出 w2.grad 与传给下一层的 h1.grad；
4. 继续反向，用 h1.grad 与 w1 算出 w1.grad.

只有 requires_grad=True 的张量梯度会被保存，其余中间结果默认释放以省内存。

前向、反向与汇总

对一个 $[B, D] \times [D, D]$ 的层，前向需要两次矩阵乘法规模的计算，反向要分别对输入和参数求梯度，因此：

$$\text{前向} = 2BD^2, \quad \text{反向} = 2BD^2 + 2BD^2 = 4BD^2$$

反向的计算量是前向的 2 倍。把网络中所有参数算上，汇总为

$$\text{总 FLOPs} \approx 6 \times (\text{数据点数量}) \times (\text{参数量})$$

这一式子适用于 MLP；对于短上下文长度的 Transformer，该近似也同样适用。

优化器谱系

- **Momentum** = SGD + 梯度的指数移动平均;
- **AdaGrad** = SGD + 按梯度平方累积进行缩放;
- **RMSProp** = AdaGrad + 梯度平方的指数移动平均;
- **Adam** = RMSProp + Momentum.

一步优化的流程: `loss.backward()` 算出梯度 → `optimizer.step()` 更新参数 → `optimizer.zero_grad(set_to_none=True)` 释放梯度内存.

`train_loop()`

一次完整的训练迭代包含: 清零梯度、前向传播、计算损失、反向传播、更新参数.

```
for x, y in data_loader:
    loss = F.cross_entropy(model(x), y)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()
```

More Memory Optimizations

梯度累积 (Gradient accumulation)

显存装不下目标 batch size 时, 把大 batch 拆成多个 micro-batch: 分别前向 / 反向并累积梯度, 累积若干步后再更新一次参数, 从而用有限显存模拟更大的 batch. 为让梯度等价于一个大 batch, 通常把每个 micro-batch 的 loss 除以累积步数 `accum_steps`.

```
loss = compute_loss(model(x), y) / accum_steps
loss.backward()
if (i + 1) % accum_steps == 0:
    optimizer.step()
    optimizer.zero_grad(set_to_none=True)
```

激活检查点 (Activation checkpointing)

反向传播需要前向计算的中间激活值, 它们占用大量显存. 激活检查点用计算换内存: 只保存少量检查点, 反向时重新计算中间激活. 收益是显著降低显存、支持更大的 batch 或更深的模型; 代价是额外的前向计算, 通常让单步训练变慢.

Summary: Resource Accounting

本讲小结：记住这几个数

- **显存**. 张量占用 = 元素数量 $\times$ 每个元素的字节数 (fp32 = 4, bf16 / fp16 = 2, fp8 = 1);
- **训练计算量**. 约 $6 \times (\text{数据点}) \times (\text{参数量})$ FLOPs: 前向 $2ND$, 反向 $4ND$;
- **FLOPs 与 FLOP/s**. 前者是计算量, 后者是硬件速度, 不要混淆;
- **算术强度**. flops/bytes; 加速器强度 = 峰值 FLOP/s $\div$ 内存带宽;
- **瓶颈判定**. 算术强度 $<$ 加速器强度 $\Rightarrow$ 内存受限, 否则为计算受限;
- **训练与推理**. 训练以大规模 matmul 为主, 是计算受限; 推理 (矩阵-向量) 是内存受限;
- **MFU**. $\min(1, \text{算术强度/加速器强度})$, 通常 ≥ 0.5 已相当不错.

常用数字速查

项目	数值
H100 峰值 (bf16, 无稀疏)	约 990 万亿 FLOP/s
H100 内存带宽	3.35 TB/s
fp32 / bf16 / fp8 每元素	4 / 2 / 1 字节
训练 FLOPs	$\approx 6 \times \text{数据点} \times \text{参数量}$
训练 GPT-3	3.14×10^{23} FLOPs
训练 GPT-4 (推测)	2×10^{25} FLOPs